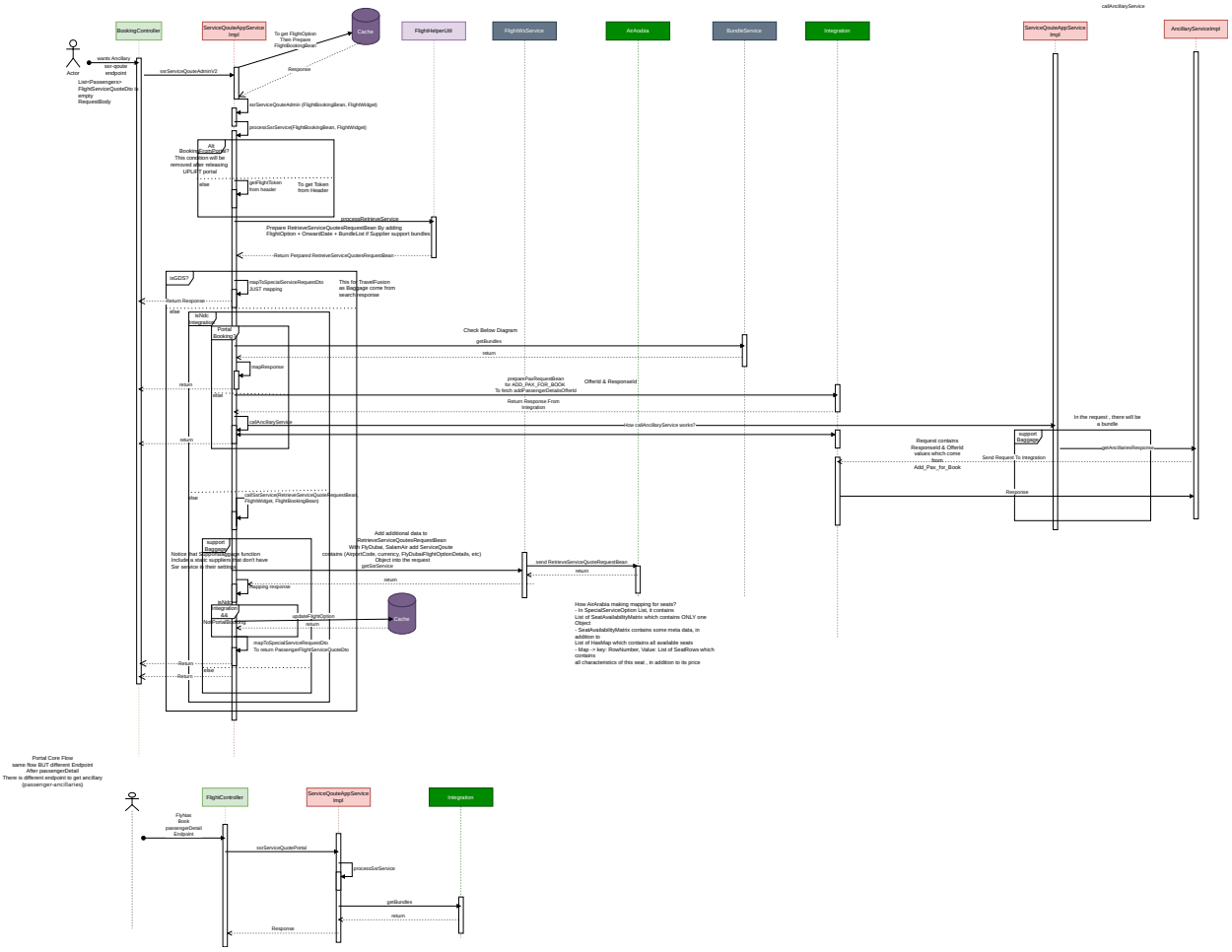
Retrieve Seat Map / SSR Ancillary / Booking Flow

Retrieve SSR-QOUTE



1. What This Diagram Covers

This is a sequence diagram for retrieving **SSR (Special Service Request) / ancillary data — including the seat map** — for a list of passengers on an existing flight booking. The entry point is a `ssr-quote` endpoint. From there, the request fans out through an orchestration service, and depending on booking type (Portal

vs not), integration type (GDS vs NDC vs LCC direct-connect), and supplier capability (baggage/SSR support), it either:

- returns quickly using data already available (GDS/TravelFusion case), or
- fetches bundle offers, or
- calls out to a generic "Ancillary Integration" system, or
- calls a specific supplier web service (e.g. AirArabia, FlyDubai, SalamAir) to get the live seat map and updates the cache with the result.

2. Participants (Lifelines)

Lifeline	Role
Actor	External caller (client / front-end) that wants ancillary/seat data
BookingController	REST controller — entry point, exposes the `ssr-quote` endpoint
ServiceQouteAppService	Main orchestration/application service — owns `ssrServiceQouteAdminV2` → `ssrServiceQouteAdmin` → `processSsrService`, i.e. almost the entire business logic in this flow
FlightHelperUtil	Utility service — builds the `RetrieveServiceQuote(s)RequestBean` used to call the supplier
Cache (×2 in diagram)	Redis-style cache — read early to fetch a cached `FlightOption`, written later via `updateFlightOption`
FlightWsService	Web-service layer that talks to airline/supplier integrations
AirArabia	Example concrete supplier integration (LCC direct-connect)
BundleService	Returns bundle offers (used only for Portal bookings)
AddPaxRequestMapper	Prepares the "Add Passenger" request bean for `ADD_PAX_FOR_BOOK`

Lifeline	Role
Integration (red ellipse)	Generic external ancillary-provider call, used in the NDC + non-Portal path

3. Entry Point & Request Shape

- **Endpoint:** `ssr-quote`
- **Actor → BookingController:** "wants Ancillary"
- **Request body:**
  - `List<Passengers>`
  - `FlightServiceQuoteDto` (sent empty by the caller — populated server-side)

4. Step-by-Step Flow

4.1 Controller → Service Entry

1. `BookingController` receives the request and calls `ServiceQouteAppService.ssrServiceQouteAdminV2(...)`.
2. This delegates into `ssrServiceQouteAdmin(FlightBookingBean, FlightWidget)`.
3. Inside `ssrServiceQouteAdmin`, the service hits the **Cache** to fetch the `FlightOption`, then uses it to prepare the `FlightBookingBean`.
4. It then calls `processSsrService(FlightBookingBean, FlightWidget)` — this is the main method and its activation spans almost the entire rest of the diagram; everything below happens inside it.

4.2 Token & "Booking From Portal" Check

- `processSsrService` first calls `getFlightToken` (from the request header).
- **Decision —** **`BookingFromPortal?`**
  - Diagram note: *"This condition will be removed after releasing UPLIFT portal."* — i.e. this is explicitly flagged as **temporary/legacy logic**.
  - The branches determine how the token is read from the header depending on whether the booking originated from the Portal.

4.3 Decision — `isGDS?`

If GDS (yes):

- Note in diagram: *"This for TravelFusion, as Baggage comes from search response."* — meaning for GDS suppliers (TravelFusion is the example), baggage/ancillary info was already returned at search time.
- The service just runs `mapToSpecialServiceRequestDto` ("**JUST** mapping" — no extra supplier call needed).
- **Returns response directly to `BookingController`.** ✅ (early exit)

If not GDS (else): → proceeds to the next decision.

4.4 Decision — `isNdc Integration?`

(nested inside the "not GDS" branch)

Branch A — `isNdc` = true

Contains a further nested decision, `Portal Booking?`:

- **If Portal Booking:**
  1. Call `BundleService.getBundles`.
  2. `mapResponse`.
  3. **Return directly to `BookingController`.** ✅ (early exit)
- **If not Portal Booking:**
  1. Call `AddPaxRequestMapper.preparePaxRequestBean(...)` for `ADD_PAX_FOR_BOOK`, to fetch `addPassengerDetailsOfferId`.
  2. Call `callAncillaryService`, which invokes the generic **Integration** system (external ancillary provider, shown as a red ellipse — representing a call out + response).
  3. **Return response back to `BookingController`.** ✅ (early exit)

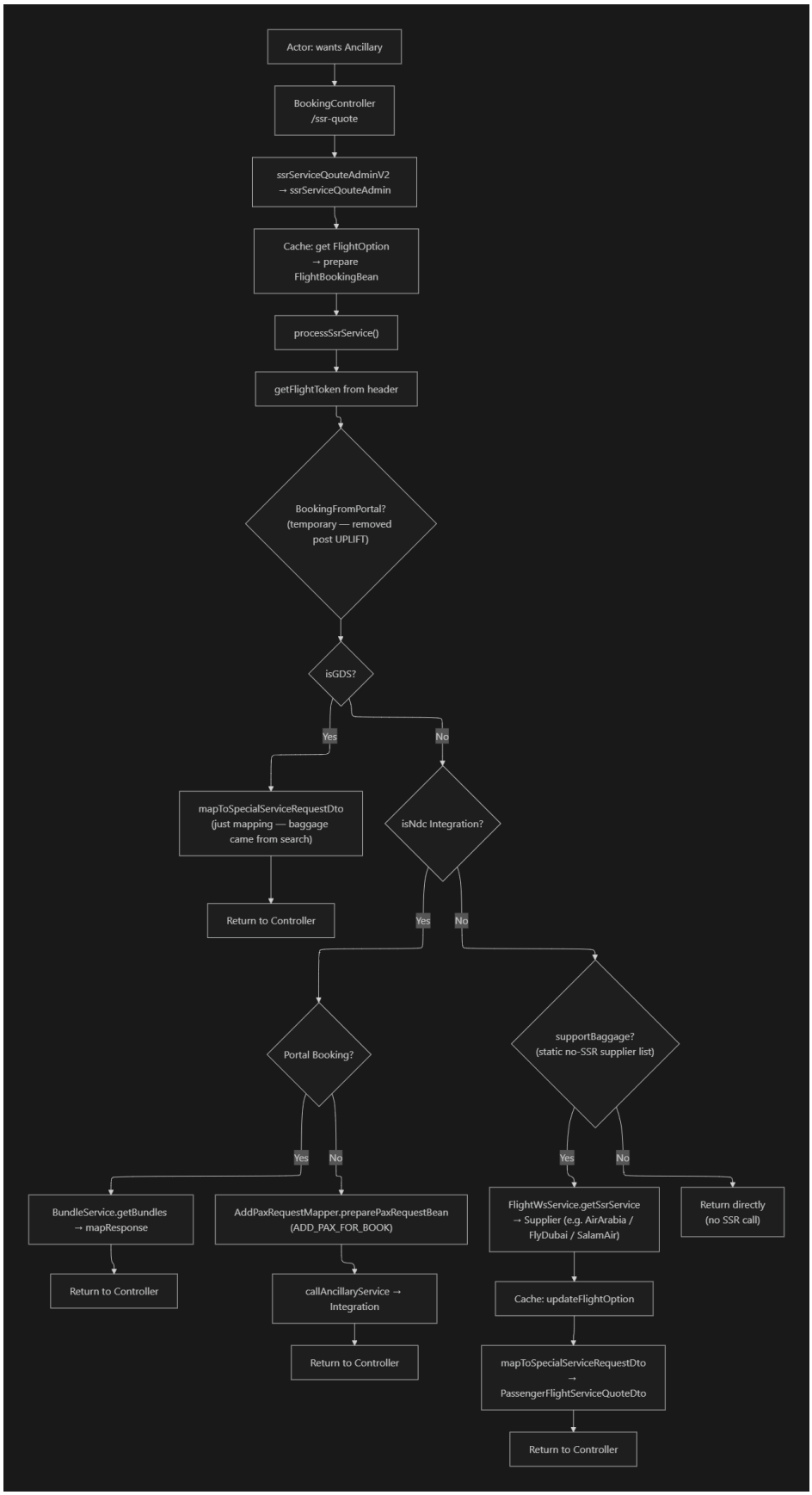
Branch B — `isNdc` = false (else)

Falls into a `supportBaggage?` decision.

> Diagram note: *"Notice that the `supportBaggage` function includes a static list of suppliers that don't have SSR service in their settings."*

- **If supplier supports baggage/SSR:**
  1. Prep step (called out specifically for **FlyDubai / SalamAir**): additional data is added to the `RetrieveServiceQuote(s)RequestBean` — `AirportCode`, `currency`, `FlyDubaiFlightOptionDetails`, etc.
  2. `ServiceQouteAppService` → `FlightWsService.getSsrService(...)`.
  3. `FlightWsService` forwards the request to the specific supplier integration — **AirArabia** is used as the worked example: `send RetrieveServiceQuoteRequestBean`.
     - **Diagram note on AirArabia's response shape (worth documenting on its own):**
       - `SpecialServiceOption` list → contains a `SeatAvailabilityMatrix` list with **only one object**.
       - `SeatAvailabilityMatrix` holds metadata **plus** a `HashMap` of all available seats.
       - Map key = `RowNumber`; value = `List<SeatRow>` (each `SeatRow` carries the seat's characteristics and price).
  4. Response returns back up through `FlightWsService` to `ServiceQouteAppService`.
  5. *(Nested condition: `isNdcIntegration && NotPortalBooking`)* → `ServiceQouteAppService` calls **Cache:** `updateFlightOption`, and the cache returns.
  6. Final mapping: `mapToSpecialServiceRequestDto` → produces `PassengerFlightServiceQuoteDto`.
  7. **Return response to `BookingController`.** ✅
- **Else (supplier is on the static no-SSR-support list):**
  - **Returns directly**, skipping the supplier call entirely. ✅ (early exit)

5. Visual Decision Tree


Actor: wants Ancillary
BookingController /ssr-quote
ssrServiceQouteAdminV2 → ssrServiceQouteAdmin
Cache: get FlightOption → prepare FlightBookingBean
processSsrService()
getFlightToken from header
BookingFromPortal? (temporary — removed post UPLIFT)
isGDS?
Yes
No
mapToSpecialServiceRequestDto (just mapping — baggage came from search)
Return to Controller
isNdc Integration?
Yes
No
Portal Booking?
Yes
No
BundleService.getBundles → mapResponse
Return to Controller
AddPaxRequestMapper.preparePaxRequestBean (ADD_PAX_FOR_BOOK)
callAncillaryService → Integration
Return to Controller
supportBaggage? (static no-SSR supplier list)
Yes
No
FlightWsService.getSsrService → Supplier (e.g. AirArabia / FlyDubai / SalamAir)
Cache: updateFlightOption
mapToSpecialServiceRequestDto → PassengerFlightServiceQuoteDto
Return to Controller
Return directly (no SSR call)

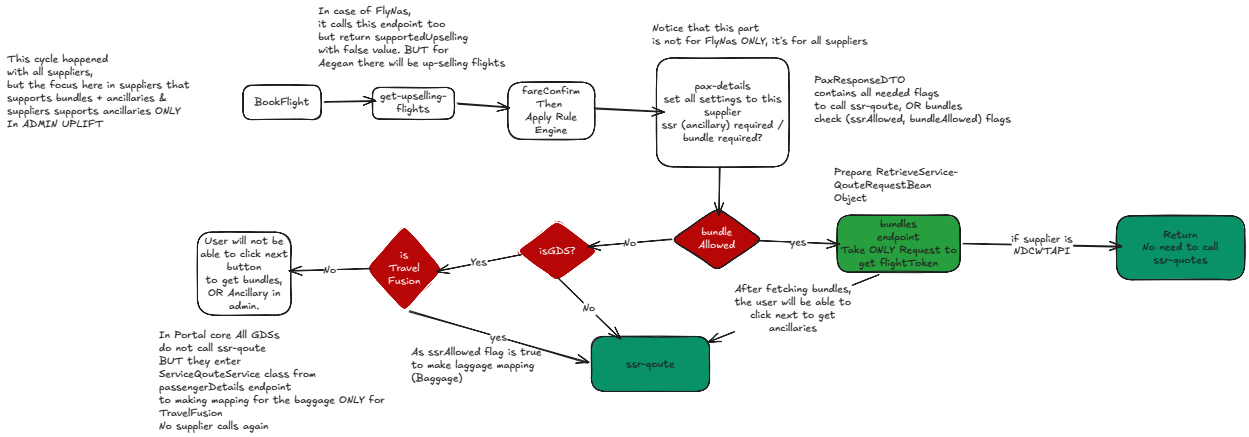
6. Exit / Return Points Summary

#	Path	What Happens	Returns
1	GDS = true	Simple mapping only (baggage already in search response)	Immediately
2	NDC + Portal Booking	`BundleService.getBundles` → map	Immediately
3	NDC + not Portal Booking	`AddPaxRequestMapper` → `callAncillaryService` → `Integration`	After external call
4	Not GDS, not NDC, supplier supports SSR	`FlightWsService.getSsrService` → supplier (e.g. AirArabia) → `updateFlightOption` in cache → map to DTO	After full supplier round-trip
5	Not GDS, not NDC, supplier on static no-SSR list	Skips the supplier call	Immediately

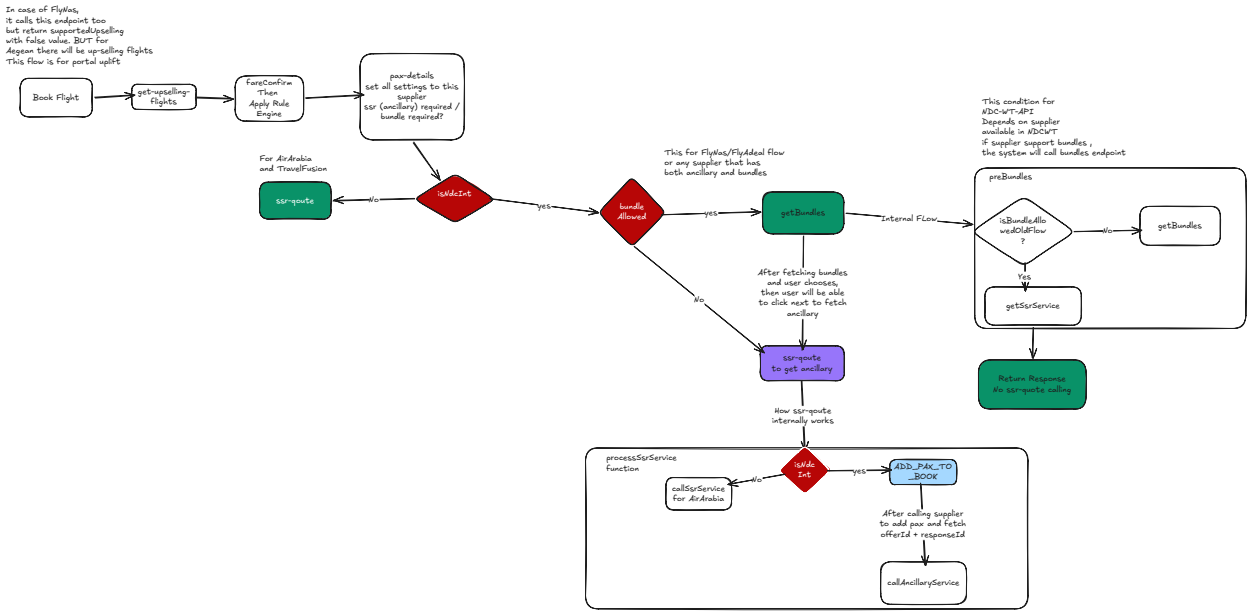
7. Notable Comments & Caveats Called Out in the Diagram

- 🟡 `BookingFromPortal?` **check is temporary** — flagged in the diagram to be removed once the UPLIFT portal is released.
- 🟡 **GDS/TravelFusion path deliberately skips a live SSR call** — baggage is assumed already present from the search response.
- 🟡 `supportBaggage` **has a static supplier exception list** — some suppliers are hardcoded as not having SSR service configured, so they short-circuit straight to return.
- 🟡 **AirArabia's seat-map shape is nested and easy to get wrong**: one `SeatAvailabilityMatrix` object holding a `Map<RowNumber, List<SeatRow>>` — worth a dedicated code comment if not already there.

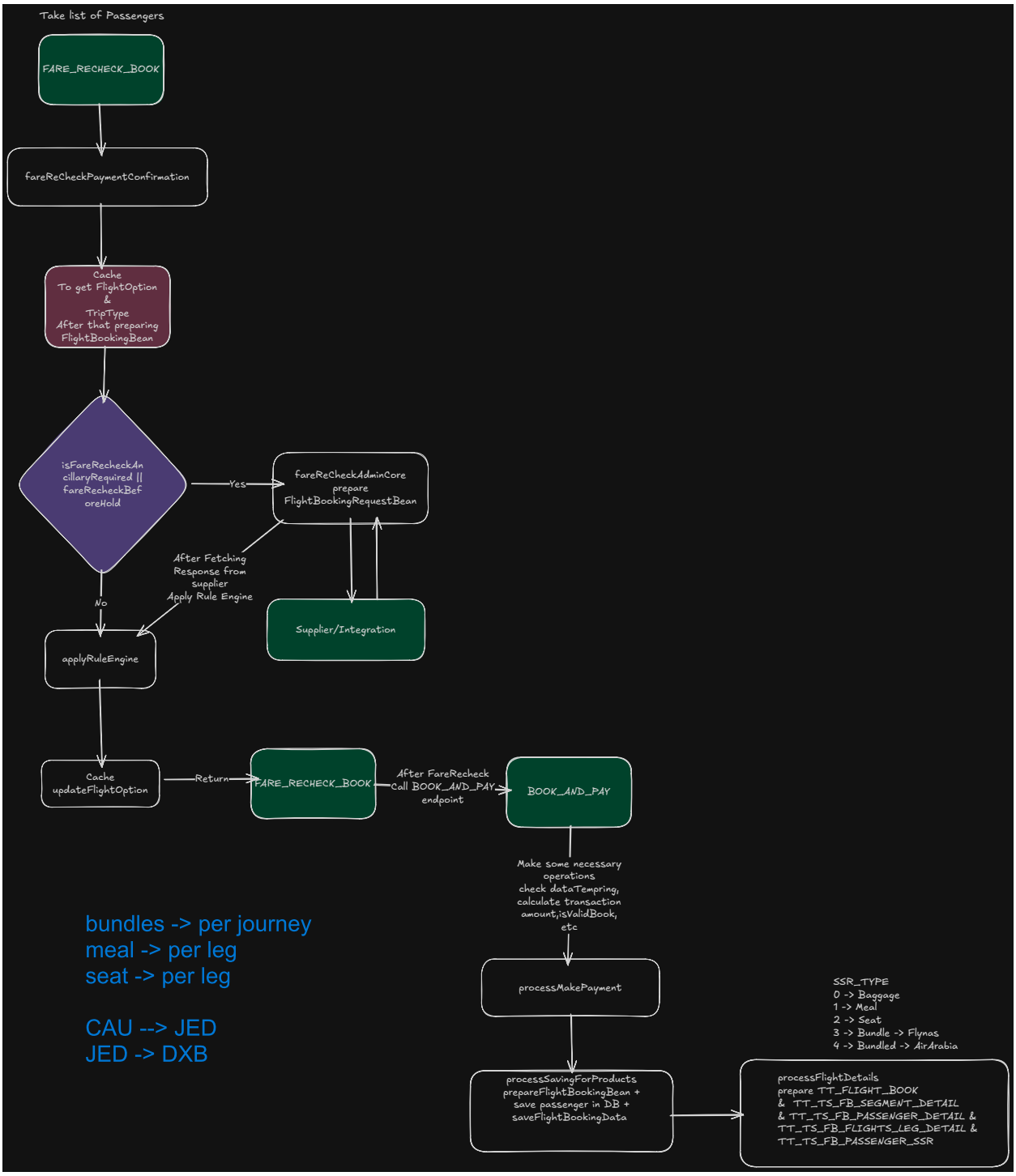
Supplier that supports Bundles and Ancillary For Admin Uplift



Supplier that supports Bundles and Ancillary For Portal Uplif

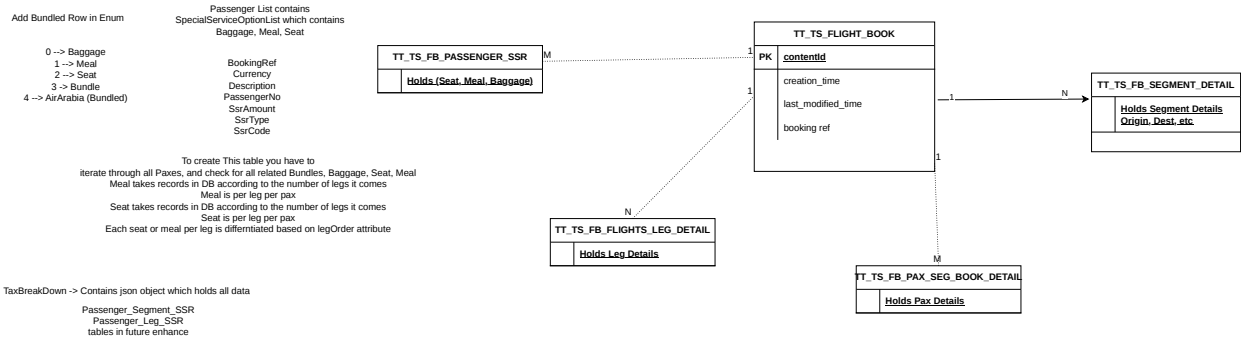


Booking Flow

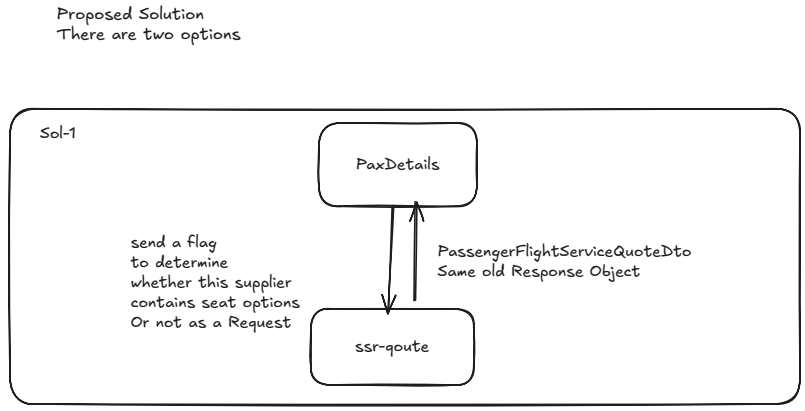


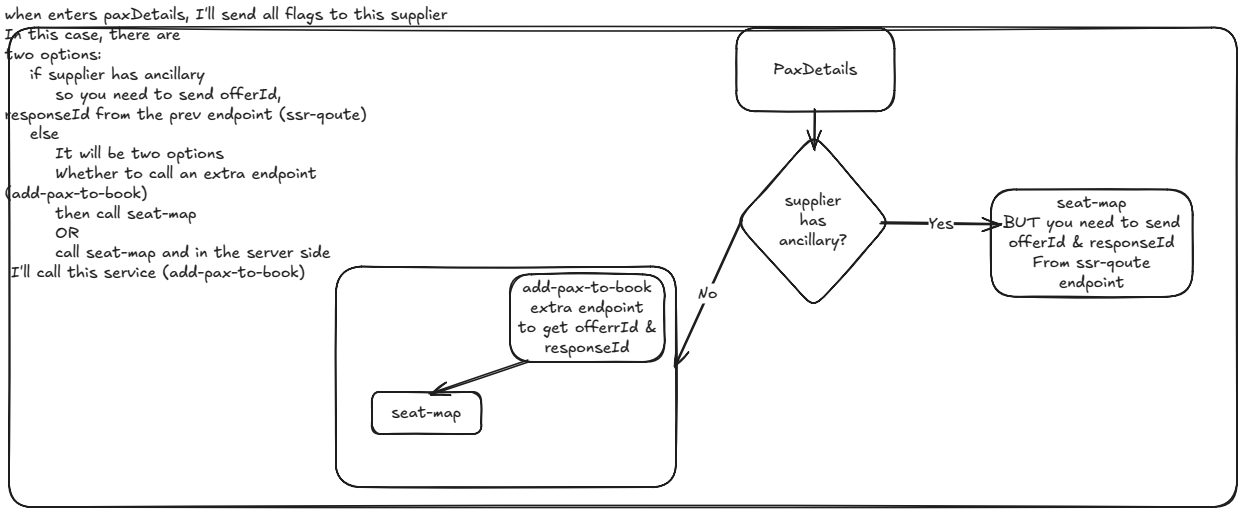
Current Schema

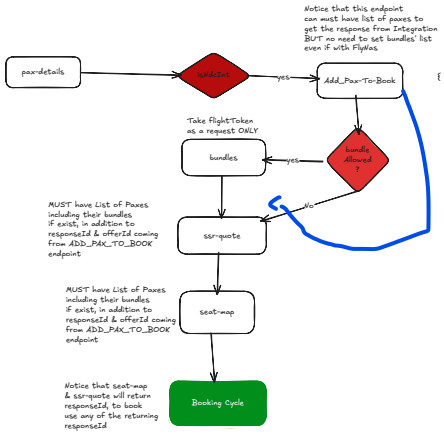
cell -> json



Proposed Solutions

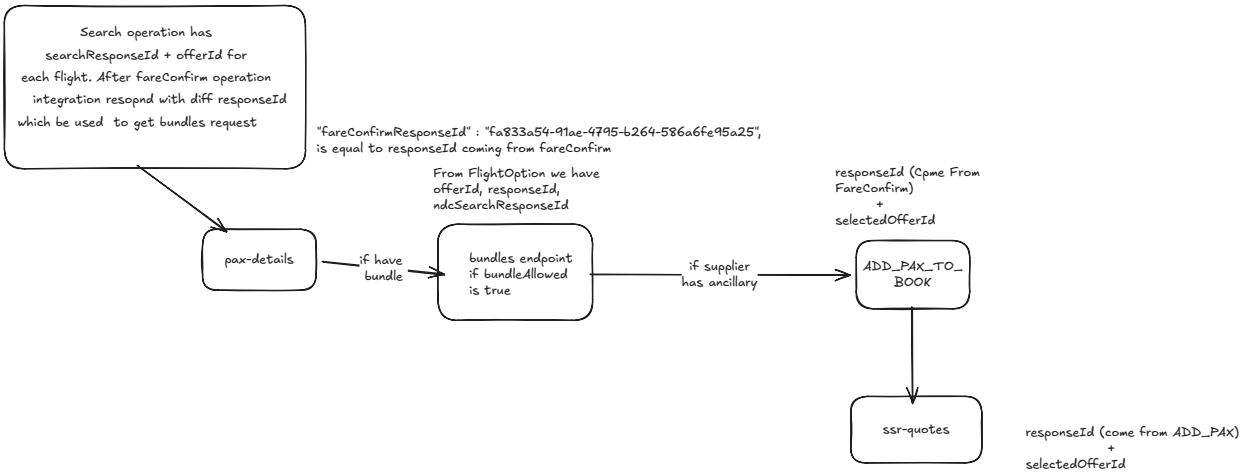






responseId
offerId


```
{
  "passengers": [
    {
      "sequence": 1,
      "passengerType": 1,
      "title": 0,
      "name": {
        "FirstName": "Sergio",
        "lastName": "Ramos"
      },
      "leadPassenger": true,
      "dob": "1997-04-24",
      "email": "test.user.ndc"@gmail.com",
      "phone": {
        "code": "+20",
        "number": "1010083491"
      },
      "document": {
        "type": 1,
        "id": "45854542RE",
        "expirationDate": "2036-02-26"
      },
      "nationality": "EG",
      "selectedPassengerId": 1570579,
      "assignedTo": null,
      "FrequentFlyer": null,
      "ticketNumber": "",
      "FlightServiceQuote": {
        "currency": "EGP",
        "FlightServiceQuote": {
          "bundles": {
            "onwardJourney": [
              {
                "origin": "CAI",
                "destination": "DXB",
                "options": [
                  {
                    "bundleName": "Plus",
                    "adultTotalPrice": 3226,
                    "childTotalPrice": 3226,
                    "infantTotalPrice": 0,
                    "includedServices": [
                      "40KG Check-in Baggage (BULK, FRAG)",
                      "BUST: ",
                      "1x7KG carry-on baggage",
                      "Hot Snack",
                      "Seat selection: All seats",
                      "Change: Free",
                      "Cancellation fee: 200SAR"
                    ],
                    "bundleMetaData": {
                      "adultTotalPrice": 3226,
                      "adultTotalTax": 0,
                      "adultFeePrice": 0,
                      "childTotalPrice": 3226,
                      "childTotalTax": 0,
                      "childFeePrice": 0,
                      "infantTotalPrice": 0,
                      "infantTotalTax": 0,
                      "infantFeePrice": 0,
                      "bundleCode": "PCAI",
                      "bundleName": "Plus",
                      "origin": "CAI",
                      "destination": "DXB",
                      "includedServices": [
                        "40KG Check-in Baggage (BULK, FRAG)",
                        "BUST: ",
                        "1x7KG carry-on baggage",
                        "Hot Snack",
                        "Seat selection: All seats",
                        "Change: Free",
                        "Cancellation fee: 200SAR"
                      ],
                      "flightId": "XY~ 264~ ~~CAI~08/20/2026 10:50~RUH~08/20/2026 13:35~~^XY~ 211~ ~~RUH~08/20/2026 15:05~DXB~08/20/2026 18:10~~&ksCrUEWBSSYlJVZbQuk=",
                      "expiresAt": 1785745165,
                      "signature": "lnlq4UFchPiF6DPlRGy4v-baIW1FC_y3jFy5NcyeS8"
                    }
                  }
                ]
              }
            ],
            "returnJourney": []
          }
        }
      }
    }
  ]
}
```

To separate ADD_PAX_TO_BOOK to be in different endpoint

What request should it takes?


```
{
  "productType" : null,
  "serviceName" : null,
  "requestBean" : null,
  "serviceConfigModel" : {
    "productName" : "Flight",
    "currencyName" : "Egyptian Pound",
    "companyName" : "EgyptLive",
    "currencyCode" : "EGP",
    "supplier" : [ {
      "pccType" : null,
      "supplierName" : "FlyNas",
      "credentialName" : "fly nas cred",
      "credential" : [ {
        "credentialName" : "NDC_URL",
        "credentialValue" : "https://ndc-supplier-integration
-staging.azurewebsites.net/",
        "subSuppl_ConversionRate" : null
      }, {
        "credentialName" : "display_name",
        "credentialValue" : "Fly Nas NDC tesssssst integratio
n",
        "subSuppl_ConversionRate" : null
      }, {
        "credentialName" : "X_API_KEY",
        "credentialValue" : "ttdb2dc2-58c5-481c-84b5-95350a3a
7978-f61360c2-f536-4b19-9a25-97b8f17ce4dc",
        "subSuppl_ConversionRate" : null
      }, {
        "credentialName" : "CLIENT_ID",
```

```
        "credentialValue" : "NDC-Core",
        "subSuppl_ConversionRate" : null
      }, {
        "credentialName" : "PROMO_CODE",
        "credentialValue" : "PR250901",
        "subSuppl_ConversionRate" : null
      } ],
      "restrictedAirlines" : [ ],
      "organizationTicketPccModels" : null,
      "dealCodes" : null,
      "paymentGatewayId" : 0,
      "paymentGatewayName" : null,
      "supplierId" : 161,
      "pccId" : 318,
      "currencyConversionRate" : 1.0,
      "supplierCurrency" : "EGP",
      "bestBuyActive" : false,
      "nearByAirportList" : null,
      "lccDealCodeMap" : null,
      "paymentMode" : 3,
      "settlementRefNumber" : "",
      "cvvNumber" : "",
      "nameOnCard" : null,
      "expMonth" : 4,
      "expYear" : 2025,
      "connectionTimeout" : "80000",
      "readTimeout" : "80000",
      "fareConfirmConnectionTimeout" : "5000",
      "fareConfirmReadTimeout" : "15000",
      "fareConfirmWithSearchConnectionTimeout" : "2000",
      "fareConfirmWithSearchReadTimeout" : "2000",
      "searchLimit" : 0,
      "displayRefundable" : true,
      "agencyCurrencyCode" : "EGP",
      "credentialType" : "LIVE",
      "countriesName" : null,
```

```
      "countriesNameSet" : [ ],
      "citiesName" : [ ],
      "pccClassification" : 1,
      "nearByStatus" : true,
      "searchAllowed" : false,
      "currencyRateFound" : true
    }, {
      "pccType" : null,
      "supplierName" : "NDCWT",
      "credentialName" : "ndcwt staging",
      "credential" : [ {
        "credentialName" : "display_name",
        "credentialValue" : "NDCWT",
        "subSuppl_ConversionRate" : null
      }, {
        "credentialName" : "X_API_KEY",
        "credentialValue" : "eb494f82eee1436ab9cd78635351fbe
b",
        "subSuppl_ConversionRate" : null
      }, {
        "credentialName" : "NDC_URL",
        "credentialValue" : "https://ndc-apis-staging-gatewa
y.azure-api.net/",
        "subSuppl_ConversionRate" : null
      } ],
      "restrictedAirlines" : [ ],
      "organizationTicketPccModels" : null,
      "dealCodes" : null,
      "paymentGatewayId" : 0,
      "paymentGatewayName" : null,
      "supplierId" : 175,
      "pccId" : 399,
      "currencyConversionRate" : 1.0,
      "supplierCurrency" : "EGP",
      "bestBuyActive" : false,
      "nearByAirportList" : null,
```

```
      "lccDealCodeMap" : null,
      "paymentMode" : 1,
      "settlementRefNumber" : null,
      "cvvNumber" : null,
      "nameOnCard" : null,
      "expMonth" : null,
      "expYear" : null,
      "connectionTimeout" : "1000",
      "readTimeout" : "80000",
      "fareConfirmConnectionTimeout" : "5000",
      "fareConfirmReadTimeout" : "15000",
      "fareConfirmWithSearchConnectionTimeout" : "2000",
      "fareConfirmWithSearchReadTimeout" : "2000",
      "searchLimit" : 0,
      "displayRefundable" : true,
      "agencyCurrencyCode" : "EGP",
      "credentialType" : "STAGING",
      "countriesName" : null,
      "countriesNameSet" : [ ],
      "citiesName" : [ ],
      "pccClassification" : 1,
      "nearByStatus" : true,
      "searchAllowed" : false,
      "currencyRateFound" : true
    } ]
  },
  "hotelCode" : null,
  "onwardFlightOption" : {
    "fareAvailabilityKey" : null,
    "trackingId" : null,
    "flightDetKey" : null,
    "flightOptionKey" : null,
    "serviceVendor" : "FlyNas",
    "totalJourneyDuration" : "145",
    "flightFare" : {
      "adultBaseFare" : 1593.0,
```

```
        "adultTax" : 3078.0,
        "adultFees" : 0.0,
        "paxTotalAmount" : 4671.0,
        "adultAllowedCheckinBaggage" : "",
        "adultAllowedHandBaggage" : "",
        "baggageIncluded" : false,
        "childBaseFare" : 0.0,
        "childTax" : 0.0,
        "childFees" : 0.0,
        "childAllowedCheckinBaggage" : "",
        "childAllowedHandBaggage" : "",
        "infantBaseFare" : 0.0,
        "infantTax" : 0.0,
        "infantFees" : 0.0,
        "infantAllowedCheckinBaggage" : "",
        "infantAllowedHandBaggage" : "",
        "currency" : "EGP",
        "fareType" : "Published Fare",
        "fareTypeCode" : null,
        "fareClass" : null,
        "rph" : null,
        "plb" : 0.0,
        "refundableInfo" : "Fly Nas NDC tesssssst integration",
        "serviceFee" : 0.0,
        "serviceTax" : 0.0,
        "tds" : 0.0,
        "totalAncillariesAmount" : 0.0,
        "totalBundlesAmount" : 0.0,
        "totalBaseFare" : 1593.0,
        "totalNet" : 0.0,
        "totalTax" : 3078.0,
        "totalFees" : 0.0,
        "transactionFee" : 0.0,
        "cabinClass" : "ECONOMY",
        "displayOnlycabinClass" : null,
        "fareSellKey" : null,
```

```
      "manualDealCode" : null,
      "corporateDealCode" : null,
      "childCorporateDealCode" : null,
      "infantCorporateDealCode" : null,
      "ancillariesAmountBreakup" : null,
      "bundlesAmountBreakup" : null,
      "paxAdtTaxBreakup" : [ {
        "taxAmount" : 957.0,
        "taxCode" : "YQ",
        "taxDesc" : "",
        "actualTaxAmount" : 957.0
      }, {
        "taxAmount" : 152.0,
        "taxCode" : "EGT",
        "taxDesc" : "",
        "actualTaxAmount" : 152.0
      }, {
        "taxAmount" : 109.0,
        "taxCode" : "EQ",
        "taxDesc" : "",
        "actualTaxAmount" : 109.0
      }, {
        "taxAmount" : 109.0,
        "taxCode" : "JK",
        "taxDesc" : "",
        "actualTaxAmount" : 109.0
      }, {
        "taxAmount" : 27.0,
        "taxCode" : "O2T",
        "taxDesc" : "",
        "actualTaxAmount" : 27.0
      }, {
        "taxAmount" : 55.0,
        "taxCode" : "O9T",
        "taxDesc" : "",
        "actualTaxAmount" : 55.0
```

```
      }, {
        "taxAmount" : 3.0,
        "taxCode" : "Q7",
        "taxDesc" : "",
        "actualTaxAmount" : 3.0
      }, {
        "taxAmount" : 1285.0,
        "taxCode" : "QH",
        "taxDesc" : "",
        "actualTaxAmount" : 1285.0
      }, {
        "taxAmount" : 109.0,
        "taxCode" : "S4",
        "taxDesc" : "",
        "actualTaxAmount" : 109.0
      }, {
        "taxAmount" : 152.0,
        "taxCode" : "XK",
        "taxDesc" : "",
        "actualTaxAmount" : 152.0
      }, {
        "taxAmount" : 11.0,
        "taxCode" : "XL",
        "taxDesc" : "",
        "actualTaxAmount" : 11.0
      }, {
        "taxAmount" : 109.0,
        "taxCode" : "E3",
        "taxDesc" : "",
        "actualTaxAmount" : 109.0
      } ],
      "paxChdTaxBreakup" : null,
      "paxInfTaxBreakup" : null,
      "paxAdtFeesBreakup" : null,
      "paxChdFeesBreakup" : null,
      "paxInfFeesBreakup" : null,
```

```
        "baggageOptionsList" : null,
        "fareNote" : null,
        "cancelPanelty" : null,
        "changePanelty" : null,
        "markupPrice" : 0.0,
        "odeysysPrice" : 4671.0,
        "serviceChargePrice" : 0.0,
        "discountPrice" : 0.0,
        "agencyMarkup" : 0.0,
        "agentMarkup" : 0.0,
        "branchMarkup" : 0.0,
        "markup" : false,
        "markdown" : false,
        "adultMarkupPrice" : 0.0,
        "adultServiceChargePrice" : 0.0,
        "adultDiscountPrice" : 0.0,
        "childMarkupPrice" : 0.0,
        "childServiceChargePrice" : 0.0,
        "childDiscountPrice" : 0.0,
        "infantMarkupPrice" : 0.0,
        "infantServiceChargePrice" : 0.0,
        "infantDiscountPrice" : 0.0,
        "actualTotalAncillariesAmount" : 0.0,
        "actualTotalBundlesAmount" : 0.0,
        "actualTotalBaseFare" : 1593.0,
        "actualTotalTaxFare" : 3078.0,
        "actualTotalFee" : 0.0,
        "actualTotalFare" : 4671.0,
        "actualSsrMealAmt" : 0.0,
        "actualSsrBaggageAmt" : 0.0,
        "actualSsrSeatAmt" : 0.0,
        "bundledServiceCharge" : 0.0,
        "actualBundledServiceCharge" : 0.0,
        "fareTypeID" : null,
        "adultFareId" : null,
        "childFareId" : null,
```

```
        "infantFareId" : null,
        "adultFareInfoRef" : null,
        "childFareInfoRef" : null,
        "infantFareInfoRef" : null,
        "adultFareType" : null,
        "childFareType" : null,
        "infantFareType" : null,
        "adultNegoFare" : false,
        "childNegoFare" : false,
        "infantNegoFare" : false,
        "adultTourCode" : null,
        "childTourCode" : null,
        "infantTourCode" : null,
        "offerID" : null,
        "offerItemID" : null,
        "paxRefID" : null,
        "paxJourneyRefID" : null,
        "bookingPcc" : null,
        "adultActualTotalFare" : 0.0,
        "childActualTotalFare" : 0.0,
        "infantActualTotalFare" : 0.0,
        "fareBrand" : "Depends on bundle choice.",
        "airpricingInfoADT" : null,
        "galileoHostTokenList" : null,
        "airPricingSolution" : null,
        "airPricingInfoMapCHD" : null,
        "airPricingInfoMapINFT" : null,
        "airPricingInfoGroupMap" : null,
        "totalAgencyCommission" : 0.0,
        "actualTotalAgencyCommission" : 0.0,
        "remark" : null,
        "travelFusionBookingFee" : 0.0,
        "actualTravelFusionFee" : 0.0,
        "travelFusionFeeDescription" : null,
        "actualTotalAdultBaseFare" : 0.0,
        "actualTotalAdultTax" : 0.0,
```

```
      "actualTotalAdultFee" : 0.0,
      "actualTotalChildBaseFare" : 0.0,
      "actualTotalChildTax" : 0.0,
      "actualTotalChildFee" : 0.0,
      "actualTotalInfantBaseFare" : 0.0,
      "actualTotalInfantTax" : 0.0,
      "actualTotalInfantFee" : 0.0,
      "actualAdultBaseFare" : 1593.0,
      "actualAdultTax" : 3078.0,
      "actualAdultFees" : 0.0,
      "actualPaxTotalAmount" : 4671.0,
      "actualChildBaseFare" : 0.0,
      "actualChildTax" : 0.0,
      "actualchildFees" : 0.0,
      "actualInfantBaseFare" : 0.0,
      "actualInfantTax" : 0.0,
      "actualInfantFees" : 0.0,
      "flightSurchargeBreakUps" : null,
      "totalSurchargeAmout" : 0.0,
      "voluntaryChangesAdult" : null,
      "voluntaryRefundsAdult" : null,
      "voluntaryChangesChild" : null,
      "voluntaryRefundsChild" : null,
      "voluntaryChangesInfant" : null,
      "voluntaryRefundsInfant" : null,
      "fareBasisCode" : null
    },
    "flightlegs" : [ {
      "ndcSegmentRef" : "NA_CAI_8/20/2026 9:40:00?AM_JED_8/2
0/2026 12:05:00?PM",
      "origin" : "CAI",
      "originName" : "Cairo International Airport",
      "destination" : "JED",
      "destinationName" : "King Abdulaziz International",
      "originCountry" : "67",
      "destinationCountry" : "195",
```

```
        "arrDate" : "2026-08-20",
        "depDate" : "2026-08-20",
        "arrTime" : "12:05:00",
        "depTime" : "09:40:00",
        "arrTerminal" : "1",
        "depTerminal" : "1",
        "bookingClass" : "E4",
        "cabinClass" : "ECONOMY",
        "carrier" : "XY",
        "carrierName" : "FLYNAS",
        "rph" : null,
        "equipment" : "320",
        "equipmentType" : null,
        "flightNumber" : "566",
        "journeyDuration" : "2h 25m",
        "numSeatsAvailable" : 0,
        "ontimeInfo" : null,
        "stopover" : "0",
        "stopoverInfo" : null,
        "validatingCarrier" : null,
        "validatingCarrierName" : null,
        "segmentCode" : null,
        "flightSwitchingWaitingTime" : "",
        "flightTime" : 145,
        "operatedByFound" : false,
        "operatedByAirline" : "XY",
        "operatedByAirlineName" : "FLYNAS",
        "originCityName" : "Cairo",
        "destinationCityName" : "Jeddah",
        "fareBasisCode" : null,
        "technicalStopList" : [ ],
        "segmentSellKey" : null,
        "inventoryLegId" : null,
        "ruleNumber" : null,
        "fareSequence" : null,
        "fareApplicationType" : null,
```

```
        "equipmentSuffix" : null,
        "legSequence" : 0,
        "journeyDurationMin" : 145,
        "flightSwitchingWaitingTimeMin" : 0,
        "segmentGroup" : null,
        "depDateTimeStr" : "2026-08-20T09:40:00",
        "arrDateTimeStr" : "2026-08-20T12:05:00",
        "providerCode" : null,
        "codeShareInfo" : null,
        "changeOfPlane" : null,
        "eTicketability" : null,
        "linkAvailability" : null,
        "optionalServicesIndicator" : null,
        "participantLevel" : null,
        "polledAvailabilityOption" : null,
        "availabilityDisplayType" : null,
        "availabilitySource" : null,
        "flightTimeStr" : null,
        "distance" : null,
        "classOfService" : "Depends on bundle choice.",
        "fareInfoRefKey" : null,
        "segmentSellKeyOld" : null,
        "originCityCode" : "CAI",
        "destinationCityCode" : "JED",
        "travelFusionClass" : null,
        "supplierClassDescription" : null,
        "cabinBaggage" : null,
        "includedBaggage" : null,
        "flightReference" : null,
        "shuraArrTime" : null,
        "shuraDeptTime" : null,
        "shuraDepDate" : null,
        "shuraArrDate" : null,
        "status" : null,
        "departDateFromResponse" : null,
        "arrivalDateFromResponse" : null,
```

```
      "supplierDepDate" : null,
      "supplierArrDate" : null,
      "resBookDesignCode" : null,
      "segmentKeyUsed" : false
    } ],
    "totalFare" : 4671.0,
    "tagFlight" : null,
    "tagFlightDesc" : null,
    "quantity" : 1,
    "origin" : "CAI",
    "destination" : "JED",
    "originId" : null,
    "destinationId" : null,
    "originCity" : "1300",
    "destinationCity" : "3635",
    "originCountry" : "67",
    "originCountryCode" : null,
    "destinationCountryCode" : null,
    "originCityCode" : "CAI",
    "destinationCityCode" : "JED",
    "destinationCountry" : "195",
    "airlinePNR" : null,
    "serviceProviderPNR" : null,
    "onwardJourneyDate" : null,
    "marketingCarrier" : "XY",
    "platingCarrier" : "XY",
    "platingCarrierName" : "FLYNAS",
    "platingAirlineType" : 2,
    "onwardJourneyDateFormat" : null,
    "airMultiAvailabilityResp" : false,
    "noOfConnectingPoints" : 0,
    "noOfTotalStops" : 0,
    "journeySellKey" : null,
    "passengerList" : null,
    "flyDubaiFlightOptionDetails" : null,
    "airIndiaExpressFlightOption" : null,
```

```
    "spiceFlightOptionDetails" : null,
    "airAirarabiaFlightOptionDetails" : null,
    "amadeusFlightOptionDetails" : null,
    "searchPCCId" : "318",
    "ticketingPCCId" : null,
    "ticketUsingSearchPCC" : false,
    "moreOptions" : null,
    "gdsOfficeId" : null,
    "supplierCurrency" : "EGP",
    "currencyRate" : 1.0,
    "baggageAllowed" : false,
    "adultCheckinBaggage" : "Depends on bundle choice.",
    "adultHandBaggage" : "Depends on bundle choice.",
    "childCheckinBaggage" : "",
    "childHandBaggage" : "",
    "infantCheckinBaggage" : "",
    "infantHandBaggage" : "",
    "includedServies" : null,
    "bundledServiceName" : null,
    "bundledServiceId" : null,
    "lccPromoCode" : null,
    "discountIdAdult" : null,
    "discountIdChild" : null,
    "discountIdInfant" : null,
    "offerId" : null,
    "offerImgPath" : null,
    "offerDesc" : null,
    "lfId" : null,
    "adtBagCode" : null,
    "chdBagCode" : null,
    "fareRule" : null,
    "ssrPrice" : 0.0,
    "offerID" : "5f6ab02f-32b5-465d-b78e-8979b2ca8bb6&msDZJDg
2YjViZjA0LTRhN2EtNGM4NC1hYTMwLTUxMGFkZjVjMDE0NpGXwNk2WFl+IDU2
Nn4gfn5DQUl+MDgvMjAvMjAyNiAwOTo0MH5KRUR+MDgvMjAvMjAyNiAxMjowN
X5+wJHcABnA2TZYWX4gNTY2fiB+fkNBSX4wOC8yMC8yMDI2IDA5OjQwfkpFRH
```

```
4wOC8yMC8yMDI2IDEyOjA1fn69MH5FNH5+WFl+RTRGQVJFfjEwMTJ+fjB+MjF
+flikIDU2NqJYWdb/aobLcNb/aobtbKNKRUSjQ0FJokVDokU0okU0pDEwMTIV
AKZFNEZBUkXCAKCiRTSgoACRmcCjSkVEo0NBSdb/aobtbNb/aobLcJTAolhZp
CA1NjbA3AAoAKMzMjDAoTHMtADAoTHMtMLCwgDA1v9qhu1s1v9qhstwwMAAAA
DAwKDAwqYzMjBNQzHCwsDDw8DAwMDAwMDAw8PDo0pFRKNDQUkAkZPAo0FEVAG
gwMDAwMA=",
    "offerItemID" : null,
    "orderID" : null,
    "noOfAdults" : 1,
    "noOfChilds" : 0,
    "noOfInfants" : 0,
    "responseID" : "ce40f536-07b0-4eb6-84d8-7656ed71ac5e",
    "ndcSearchResponseID" : "34a48f89-159e-4697-9cea-8111e770
88d6",
    "carryOnCharges" : null,
    "bookAndHoldAllowedForSupplier" : false,
    "baggageAllowedForSupplier" : true,
    "haveBundles" : true,
    "canBeHeld" : false,
    "appliedDealCode" : null,
    "appliedPromotionCode" : "",
    "offerHasAncillary" : true,
    "brandCodeSabre" : [ ],
    "fareBaseCodeSabre" : [ ],
    "fareBasisCodes" : null,
    "routingId" : null,
    "onlyBaggageFareSearch" : false,
    "flightUUID" : null,
    "rulesCommonBean" : null,
    "journeyDurationInMinutes" : 145,
    "nDCjourneyID" : "XY~ 566~ ~~CAI~08/20/2026 09:40~JED~08/
20/2026 12:05~~",
    "agencyCurrencyCode" : "EGP",
    "ndcRulesAndPenalties" : [ " ############### rules and pe
nalties for segment CAI -> JED ###############" ],
    "supplierSettings" : {
```

```
      "paxAddAllowed" : false,
      "dynamicCurrencyAllowed" : false,
      "fareAllowed" : true,
      "gdsSupplier" : false,
      "bspAllowed" : false,
      "checkSupplierAllowed" : true,
      "multiCitySupported" : true,
      "syncPnrAllowed" : false,
      "importBookedPnrAllowed" : false,
      "timeLimitHoldAllowed" : false,
      "searchUpsellingAllowed" : false,
      "ancillaryFlowAllowed" : true,
      "upSellingAllowed" : false,
      "timeLimitAllowed" : false,
      "dealCodeAllowed" : false,
      "fareConfirmBeforePayAfterHoldAllowed" : false,
      "iqamaAllowed" : false,
      "bookAndHoldAllowed" : false,
      "cancelBookAndHoldAllowed" : false,
      "ticketVoidAllowed" : false,
      "baggageAllowed" : true,
      "bundleAllowed" : true
    },
    "galileoAirReservationLocator" : null,
    "galileoUniversalRecordLocator" : null,
    "currentPnrImportVersion" : 0,
    "offerProperties" : {
      "withBaggage" : false,
      "supplierType" : null,
      "refundable" : false
    },
    "laggage" : null,
    "vendorSignature" : null,
    "connecting" : false,
    "multiCarrierForApplyRule" : false,
    "domestic" : false,
```

```
      "optionSegmentBean" : null,
      "importPnr" : false,
      "fullLaggage" : false,
      "lccflight" : true,
      "multiCarrier" : false,
      "ancillaryRequired" : false,
      "upSellingOption" : false,
      "baggageRequired" : false,
      "cardPreRegister" : false,
      "dealCodeApplied" : false,
      "seatMapRequired" : false,
      "ndcintegration" : true,
      "fareCheckAncillariesRequired" : false,
      "mealRequired" : false,
      "viaFlight" : false,
      "validLaggage" : false,
      "useTFPrepay" : false,
      "promoFare" : false,
      "hazUmarhJed" : false,
      "promoted" : false,
      "orderItemID" : null
    },
    "ticketUsingSearchPCC" : false,
    "ticketingPCCId" : null,
    "searchPCCId" : "318",
    "passengerList" : [ {
      "title" : "Mr",
      "firstName" : "Sergio",
      "lastName" : "Ramos",
      "middleName" : null,
      "dob" : "1997-04-29",
      "age" : 29,
      "paxSequence" : 1,
      "paxId" : null,
      "companyId" : "",
      "gender" : "Male",
```

```
      "frequentFlyerNumber" : "",
      "type" : "ADT",
      "email" : "test.user.ndc@gmail.com",
      "contactNumber" : "1010083991",
      "tourCode" : "",
      "dealCode" : "",
      "nationality" : "EG",
      "nationality3code" : "",
      "visa" : "",
      "passportNumber" : "45854542RE",
      "document" : {
        "documentId" : "45854542RE",
        "documentType" : "Passport",
        "documentExpiry" : "2036-02-26"
      },
      "passportDateOfIssue" : "",
      "passportIssuingCountry" : "GBR",
      "passportDateOfExpiry" : "2036-02-26",
      "agentId" : "",
      "additionalSegmentInfo" : "",
      "profileId" : "",
      "travellingWithPaxId" : "1570579",
      "address" : null,
      "contactInfos" : null,
      "leadPassenger" : true,
      "frequentFlyerNumberMap" : null,
      "ticketNumber" : "",
      "ticketSupplierCarrierCode" : null,
      "ticketRPHNumber" : null,
      "assignedTo" : null,
      "baggageServiceOpted" : false,
      "mealServiceOpted" : false,
      "seatMapOpted" : false,
      "bundleOpted" : true,
      "passport" : null,
      "passengerTattoNo" : null,
```

```
    "passengerNMNo" : null,
    "paxUniqueRefId" : null,
    "phoneCode" : "+20",
    "nationalityName" : "Egypt",
    "assingedInfant" : null,
    "personOrgId" : "",
    "lapChildId" : "",
    "passengerActiveStatus" : "1",
    "passengerAccompany" : "",
    "passengerRefID" : null,
    "passenger" : null,
    "specialServiceOption" : [ {
      "serviceType" : null,
      "segmentType" : "Onward",
      "serviceID" : null,
      "logicalFlightID" : 0,
      "departureDate" : null,
      "amount" : null,
      "overrideAmount" : false,
      "currencyCode" : "EGP",
      "commissionable" : null,
      "refundable" : null,
      "chargeComment" : null,
      "ssrName" : null,
      "ssrCode" : null,
      "description" : null,
      "specialSeviceOptionMulticities" : null,
      "baggageList" : null,
      "seatMatrixList" : null,
      "mealOptionList" : null,
      "bundleOptionList" : [ {
        "flightId" : "XY~ 566~ ~~CAI~08/20/2026 09:40~JED~08/
20/2026 12:05~~&ksCrUENBSSYlJVZDQUk=",
        "origin" : "CAI",
        "destination" : "JED",
        "bundleCode" : "PCAI",
```

```
        "bundleName" : "Plus",
        "adultTotalPrice" : 2652.0,
        "adultTotalTax" : 0.0,
        "adultFeePrice" : 0.0,
        "childTotalPrice" : 0.0,
        "childTotalTax" : 0.0,
        "childFeePrice" : 0.0,
        "infantTotalPrice" : 0.0,
        "infantTotalTax" : 0.0,
        "infantFeePrice" : 0.0,
        "includedServices" : [ "40KG Check-in Baggage (BULK,
FRAG)", "BNST: ", "1x7KG carry-on baggage", "Hot Snack", "Sea
t selection: All seats", "Change: Free", "Cancellation fee: 2
00SAR" ]
      } ],
      "actualAmount" : 0.0,
      "physicalFlightId" : null,
      "optionId" : null,
      "flightId" : null,
      "passengerRefId" : null,
      "origin" : null,
      "destination" : null,
      "baggagePerSegment" : null,
      "serviceValue" : null,
      "codeType" : null,
      "quantityAvailable" : 0,
      "ssrcategory" : 0
    } ],
    "ancillaries" : null,
    "wbcid" : ""
  } ]
}
```


Client-Server Contract

What is inside the response from integration?

- FlightSegments
- PassengerDetails
- SeatMapDetails → It contains data for all available seats per segment, so if you have 3 segments, we'll have 3 records in the map (key → value), (SEG1 → SeatAvailability)
    - CabinLayoutsByDeck → if a plane has two stairs (Main - Upper)
        - Main → Zones
        - Upper → Zones
    - What Zones?
        - The type of the areas in the plane
        - Zones holds metaData → zoneId + List of zones' values
    - Rows → Holds Seats and their characteristics + prices
        - RowMetaData
        - Cells
            - CellMetaData
            - List of CellDetails
                - CellDetailsMetaData
                - Map of Map → contains seatPriceDetailsPerPax
            - List of Cell Characteristics which contains CellCharacteristicsMetaData
- PassengerSeatMapAvailability
    - Map of Map
        - KeyOfMap1 → represents PAX1
        - KeyOfMap2 → represents SEG1
        - ValueOfMap1 → Is the other map

- ValueOfMap2 → List of seatCode

How contract will be?

Returns the complete seat map for **all flight segments** in the booking session.

Supports:

- One-way
- Round Trip
- Multi-city itineraries

A single request returns every segment.

Success Response


```
{
  "supplier": "AirArabia",
"currency":"AED",
"passengers": [
  {
  "passengerId":"PAX1",
  "type":"ADULT",
  "label":"Ahmed Al Sayed"
  },
  {
  "passengerId":"PAX2",
  "type":"CHILD",
  "label":"Layla Al Sayed"
  }
],
"segments": [
{
"segmentId":"SEG1",
"origin":"DXB",
"destination":"LHR",
"flightNumber":"G9780",
"departureDateTime":"2026-08-14T09:15:00",
"arrivalDateTime":"2026-08-14T13:40:00",
"cabinLayout": {
"decks": [
{
"deck":
```

```
            "MAIN",
            "zones": [
              {
                "zoneId":"Z1",
                "columns": [
                  {
                    "colId": 1,
                    "label": "A",
                    "zoneType": "SEAT"
                  },
                  {
                    "colId": 2,
                    "label": "B",
                    "zoneType": "SEAT"
                  },
                  {
                    "colId": 3,
                    "label": "AISLE/LAVATORY",
                    "zoneType": "AISLE/LAVATORY"
                  }
                ]
              }
            ]
          }
        ]
      },
      "rows": [
        {
          "rowNumber":1,
          "deck":"MAIN",
          "zoneId":"Z1",
          "cells": [
            {
              "cellCategory": "SEAT",
              "cellId":"SEG1-1A",
              "cellCode":"1A",
              "column":"A",
              "cellAvailability":"AVAILABLE_PAID",
              "cellCharacteristics": ["WINDOW","LIE_FLAT"],
              "seatPriceDetailsByPassenger": {
                "PAX1": 25.00,
                "PAX2": 30.00
              }
            }
          ]
        }
      ],
      "passengerEligibility": {
        "PAX1": ["1A","1C","20A"],
        "PAX2": ["20A","20B"]
      }
```

```
    }
  ]
}
```

2. FareRecheck Request DTO

Fare Recheck Request Contract & Validation

Purpose

The **Fare Recheck** endpoint verifies that the seats selected by the client are still valid before proceeding to payment.

Since this endpoint is responsible for confirming pricing and availability, **the backend never trusts any seat information received from the client**.

Instead, every request is validated against the **server-side seat map snapshot** that was generated during the Seat Map API response. Only after all validations succeed is the supplier contacted for a real-time fare and seat availability check.

Request Contract

Request Body

```
{
  "supplier":"AirArabia",
  "credentialsSelector":"Default",
  "responseId":"3f9a1b2c-7d4e-4a6f-9c2b-8e1d5f0a6b31",
  "selectedOfferId":"OFR-AA-20260726-0091",
  "selectedSeats": [
    {
      "passengerId":"PAX1",
      "segmentId":"SEG1",
      "cellId":"SEG1-12A"
    },
    {
      "passengerId":"PAX2",
```

```
      "segmentId":"SEG1",
      "cellId":"SEG1-12B"
    }
  ]
}
```

Proposed response contract:

json

```
{
  "currency": "AED",
  "priceChanged": true,
  "passengers": [
    {
      "passengerRefId": "PAX1",
      "totalSeatAmount": 25.00,
      "seats": [
        {
          "code": "12A",
          "status": "CHANGE/NOT_CHANGED",
          "originalPrice": 20.00,
          "currentPrice": 25.00,
          "priceChanged": true,
          "features": ["WINDOW", "EXIT_ROW"],
          "description": "Window seat, exit and emergency exi
t"
        }
      ]
    }
  ]
}
```

Request Fields

Field	Type	Required	Description
`supplier`	String	✅	Supplier code (e.g. `AirArabia`).
`credentialsSelector`	String	✅	Supplier credential set used for the supplier recheck.
`responseId`	UUID	✅	Identifier of the cached Seat Map snapshot. Used as the server-side source of truth.
`selectedOfferId`	String	✅	Offer selected by the client. Must match the offer stored inside the cached snapshot.
`selectedSeats`	Array	✅	List of all selected seats. One record per passenger and segment.

selectedSeats Object

Field	Type	Description
`passengerId`	String	Passenger identifier from the cached seat map response.
`segmentId`	String	Flight segment identifier.
`cellId`	String	Selected seat identifier.

Validation Overview

Validation happens in **two consecutive layers** before any supplier API is called.

```
Client Request
      |
      ▼
Layer 1
Structural Validation
      |
      ▼
Layer 2
Snapshot Validation
```

```
      |
      ▼
Supplier Fare Recheck
      |
      ▼
Return Latest Price & Availability
```

If **either layer fails**, processing stops immediately and **no supplier request is made**.

Layer 1 — Structural Validation

This layer performs **cheap, stateless validation** using only the request body.

No cache access is required.

Validation Rules

Rule	Validation
`responseId` must not be blank	Reject if missing or empty
`selectedOfferId` must not be blank	Reject if missing or empty
`selectedSeats` must not be empty	Reject if array is missing or contains zero items
No duplicate `(passengerId, segmentId)`	One passenger may select only one seat per segment
No duplicate `cellId`	Two passengers cannot select the same seat in the same request

---

Layer 1 Flow

```
Request Received
        |
        ▼
responseId exists?
        |
   No ──────► Reject
        |
       Yes
```



Layer 2 — Trust Boundary Validation

This layer validates the request against the **server-side cached Seat Map snapshot** referenced by `responseId`.

The cached snapshot is the **only trusted source of truth**.

Validation Sequence

1. Resolve responseId

Verify that `responseId` maps to a valid cached Seat Map snapshot.

If not:

> RESPONSE_EXPIRED

Reason:

- Cache expired
- Unknown responseId
- Seat map was never generated

---

2. Validate Offer

Verify:

```
selectedOfferId
      ==
cached.offerId
```

If not:

> OFFER_MISMATCH

---

3. Validate Segment

Ensure:

```
segmentId
```

exists inside the cached Seat Map.

Otherwise:

> INVALID_SEGMENT

---

4. Validate Seat

Ensure:

```
cellId
```

exists inside the cached segment.

Otherwise:

> SEAT_NOT_FOUND

---

5. Validate Passenger

Ensure:

```
passengerId
```

exists inside the cached passenger list.

Otherwise:

> UNKNOWN_PASSENGER

---

6. Validate Passenger Eligibility

For the given segment:

```
cachedPassengerEligibility
```

must contain

```
cellId
```

Otherwise:

> SEAT_NOT_ELIGIBLE_FOR_PASSENGER

Examples:

- Exit row selected by a child
- Restricted mobility seat
- Crew-only seat

- Premium seat not eligible

---

7. Validate Cached Availability

The cached seat status must be one of:

- `AVAILABLE_FREE`
- `AVAILABLE_PAID`

Reject if the cached status is:

- OCCUPIED
- BLOCKED
- UNAVAILABLE
- RESERVED
- Any other non-selectable state

Error:

> SEAT_NOT_AVAILABLE

Layer 2 Flow

```
Load Snapshot(responseId)
          |
          ▼
Snapshot Exists?
          |
 No ──────────► RESPONSE_EXPIRED
          |
         Yes
          ▼
Offer Matches?
          |
 No ──────────► OFFER_MISMATCH
          |
         Yes
          ▼
Segment Exists?
          |
```

```
 No ──────────► INVALID_SEGMENT
          |
         Yes
          ▼
Seat Exists?
          |
 No ──────────► SEAT_NOT_FOUND
          |
         Yes
          ▼
Passenger Exists?
          |
 No ──────────► UNKNOWN_PASSENGER
          |
         Yes
          ▼
Passenger Eligible?
          |
 No ──────────► SEAT_NOT_ELIGIBLE_FOR_PASSENGER
          |
         Yes
          ▼
Seat Available?
          |
 No ──────────► SEAT_NOT_AVAILABLE
          |
         Yes
          ▼
Supplier Fare Recheck
```

Failure Response

When validation fails, the backend returns a typed error with detailed information.

```
{
  "errorCode":"SEAT_NOT_ELIGIBLE_FOR_PASSENGER",
  "message":"One or more selected seats failed validation.",
  "details": [
    {
      "passengerId":"PAX2",
      "segmentId":"SEG1",
      "cellId":"SEG1-12A",
      "reason":"SEAT_NOT_ELIGIBLE_FOR_PASSENGER"
    }
```

```
  ]
}
```

Error Codes

Error Code	Description
`RESPONSE_EXPIRED`	Cached Seat Map snapshot no longer exists or has expired.
`OFFER_MISMATCH`	Selected offer does not match the cached offer.
`INVALID_SEGMENT`	Segment ID is not present in the cached snapshot.
`SEAT_NOT_FOUND`	Selected seat was never part of the offered Seat Map.
`UNKNOWN_PASSENGER`	Passenger does not exist in the cached passenger list.
`SEAT_NOT_ELIGIBLE_FOR_PASSENGER`	Passenger is not allowed to select the requested seat.
`SEAT_NOT_AVAILABLE`	Seat was not selectable in the cached snapshot.
`DUPLICATE_SEAT_SELECTION`	Two passengers selected the same seat.

What Validation Guarantees

After both validation layers succeed, the backend knows that:

- ✅ The request references a valid Seat Map response.
- ✅ The selected offer matches the cached offer.
- ✅ Every passenger exists.
- ✅ Every segment exists.
- ✅ Every seat was actually offered.
- ✅ Every passenger is eligible for the selected seat.
- ✅ Every seat was selectable when the Seat Map was generated.

At this point, the request is considered **internally consistent** and safe to send to the supplier.

What Validation Does Not Guarantee

Validation **does not** guarantee that the seat is still available.

Between Seat Map retrieval and Fare Recheck:

- another customer may reserve the seat,
- pricing may change,
- the supplier may remove the seat.

Only the **supplier Fare Recheck response** is authoritative for the current seat availability and pricing.

Concurrency Considerations

Layer 2 validates against a cached snapshot and **does not lock seat inventory**.

As a result:

- Multiple requests referencing the same `responseId` can pass validation simultaneously.
- Both requests may proceed to the supplier.
- The supplier is responsible for determining which request succeeds.

This behavior is expected and does **not** indicate a validation failure.



The supplier's response remains the final authority on seat availability.